Lapa Caçambas — Documentação Sprint 1

Gerado em: 30/03/2026 **Responsável:** Rodrigo **Parceira:** Nathalia (fiscal/financeiro) **Stack:** Node.js + Express + TypeScript + Supabase + React + Vite + Shadcn/UI

---

1. Visão Geral da Sprint

A Sprint 1 cobriu a fundação do sistema: autenticação, controle de acesso por papéis (RBAC) e os módulos de cadastro-base — clientes, obras, endereços e contatos. Além disso, todo o backend da Sprint 2 (ativos: caçambas, veículos, máquinas, motoristas) foi antecipado e entregue.

Status das Entregas

Item	Sprint	Status
RBAC / Auth completo	1	✅ Entregue
Schema das tabelas base	1	✅ Já existia nas migrations
CRUD Clientes	1	✅ Entregue
CRUD Obras	1	✅ Entregue
CRUD Endereços de Entrega	1	✅ Entregue
CRUD Contatos do Cliente	1	✅ Entregue
`shared/enums.ts` — fonte única	1	✅ Entregue
`shared/contracts.md` — contratos	1	✅ Entregue
CRUD Caçambas + Unidades	2	✅ Antecipado
CRUD Veículos	2	✅ Antecipado
CRUD Máquinas	2	✅ Antecipado
CRUD Motoristas	2	✅ Antecipado

---

2. Arquitetura do Backend

O backend segue uma arquitetura em três camadas, separando responsabilidades claramente:

```
HTTP Request
     ↓
Controller  (traduz HTTP ↔ dados limpos, valida IDs, retorna status codes)
     ↓
Service     (lógica de negócio, acessa o banco, converte snake_case → camelCase)
     ↓
Supabase    (banco de dados PostgreSQL gerenciado)
```

Cada módulo tem exatamente três arquivos:

Arquivo	Responsabilidade
`*.types.ts`	Contratos de dados: Row (banco), Dto (frontend), CreateDto, UpdateDto
`*.service.ts`	Lógica de negócio, queries Supabase, função `toDto()`
`*.controller.ts`	Rotas Express, validação de parâmetros HTTP, status codes

Por que essa separação?

- O **service** não sabe nada sobre HTTP — pode ser reutilizado por outros serviços, workers, scripts
- O **controller** não tem lógica de negócio — apenas traduz e delega
- Os **types** garantem que banco e frontend "falem a mesma língua" com segurança de tipos em tempo de compilação

---

3. Autenticação e RBAC

Fluxo de Autenticação

```
Login (email + senha)
       ↓
Supabase Auth → JWT (access_token + refresh_token)
       ↓
Frontend armazena tokens
       ↓
Cada request → Header: Authorization: Bearer <access_token>
       ↓
requireAuth() → verifica JWT + consulta user_roles no banco
       ↓
Acesso liberado ou 401/403
```

Middleware `requireAuth`

```
// Uso no app.ts
app.use('/api/clientes',
  requireAuth(['administrador', 'atendimento', 'gestor', 'fiscal']),
  clientesRouter
);
```

O middleware:

1. Extrai o token do header `Authorization: Bearer`
2. Valida com Supabase (`supabase.auth.getUser(token)`)
3. Busca os roles do usuário em `user_roles`
4. Verifica se o usuário tem pelo menos um dos roles permitidos
5. Injeta `req.user` com `{ id, email, roles }`

Tabelas de Autenticação

Tabela	Descrição
`auth.users`	Gerenciada pelo Supabase Auth
`public.profiles`	Dados extras: nome, email, criado automaticamente via trigger
`public.user_roles`	Relação usuário ↔ role (pode ter múltiplos roles)

Roles do Sistema

Role	Permissões
`administrador`	Acesso total a todos os módulos
`gestor`	Gestão operacional completa
`atendimento`	Clientes, obras, pedidos
`operador`	Ativos e execução de pedidos
`fiscal`	Notas fiscais e faturamento
`financeiro`	Boletos e contas a pagar/receber
`motorista`	App mobile (implementação futura)

4. Fonte Única de Enums — `shared/enums.ts`

Problema resolvido: antes, os enums estavam definidos em `src/types/enums.ts` com valores desatualizados em relação ao banco. Qualquer mudança precisava ser feita em dois lugares.

Solução: criar `shared/enums.ts` como fonte canônica, e fazer `src/types/enums.ts` ser apenas um re-export.

```
shared/
  enums.ts          ← DEFINIÇÃO (única cópia real)

src/types/
  enums.ts          ← export * from '../../shared/enums'  (shim)

backend/src/modules/clientes/
  clientes.types.ts ← import from '../../../../shared/enums'
```

Enums Definidos

Enum	Valores
`StatusCliente`	ativo, inativo, bloqueado, prospecto
`TipoCliente`	pessoa_fisica, pessoa_juridica
`StatusPedido`	rascunho, aguardando_aprovacao, aprovado, pendente_programacao, programado, em_execucao, concluido, cancelado, suspenso, faturado
`TipoPedido`	locacao, coleta, troca, remocao
`StatusCacamba`	disponivel, em_uso, manutencao, descartada
`StatusVeiculo`	disponivel, em_uso, manutencao, inativo
`StatusMaquina`	disponivel, em_uso, manutencao, inativo
`StatusMotorista`	ativo, inativo, ferias, afastado, bloqueado
`StatusExecucao`	em_andamento, concluida, cancelada
`StatusOcorrencia`	aberta, em_andamento, resolvida, cancelada
`StatusNotaFiscal`	pendente, processando, emitida, cancelada, substituida, erro
`StatusBoleto`	pendente, pago, vencido, cancelado
`TipoFluxo`	entrada, saida
`AppRole`	administrador, gestor, atendimento, operador, fiscal, financeiro, motorista

5. Módulos Implementados

5.1 Clientes — `/api/clientes`

Tabela: `public.clientes` **Roles:** administrador, atendimento, gestor, fiscal

Campo	Tipo	Notas
`id`	bigserial	PK
`nome`	text NOT NULL	
`fantasia`	text	Nome fantasia
`tipo`	TipoCliente	pessoa_fisica / pessoa_juridica
`status`	StatusCliente	Default: ativo
`cpf` / `cnpj`	text	
`telefone` , `celular` , `email`	text	
`endereco` … `estado`	text	Endereço principal
`endereco_cobranca` …	text	Endereço de cobrança
`created_by` , `updated_by`	uuid	Auditoria
`deleted_at`	timestamptz	Soft delete

Rotas:

Método	Path	Query Params
GET	`/api/clientes`	`busca` , `status` , `tipo` , `page` , `limit`
GET	`/api/clientes/:id`	—
POST	`/api/clientes`	Body: `CreateClienteDto`
PUT	`/api/clientes/:id`	Body: `UpdateClienteDto`
DELETE	`/api/clientes/:id`	Soft delete (seta `deleted_at`)

Detalhe importante: A listagem (`GET /`) filtra automaticamente registros com `deleted_at IS NOT NULL` . Clientes deletados nunca aparecem na listagem — mas o registro permanece no banco.

5.2 Contatos do Cliente — `/api/contatos`

Tabela: `public.contatos_cliente` **Roles:** administrador, atendimento, gestor, fiscal

Campo	Tipo	Notas

`id`	bigserial	PK
`cliente_id`	bigint NOT NULL	FK → clientes (CASCADE DELETE)
`nome`	text NOT NULL	
`telefone` , `celular` , `email` , `cargo`	text	
`principal`	boolean	Default: false
`created_at`	timestamptz	

> Sem `deleted_at` — delete físico é aceitável pois contatos são dados secundários do cliente.

Rotas:

Método	Path	Observação
GET	`/api/contatos?clienteId=N`	`clienteId` obrigatório
GET	`/api/contatos/:id`	
POST	`/api/contatos`	Body deve incluir `clienteId`
PUT	`/api/contatos/:id`	`clienteId` não pode ser alterado
DELETE	`/api/contatos/:id`	Delete físico

A listagem retorna contatos ordenados: principal primeiro, depois por nome.

5.3 Obras — `/api/obras`

Tabela: `public.obras` **Roles:** administrador, atendimento, gestor, fiscal

Campo	Tipo	Notas
`id`	bigserial	PK
`cliente_id`	bigint NOT NULL	FK → clientes
`nome`	text NOT NULL	
`responsavel` , `telefone`	text	
`endereco` … `estado`	text	Localização da obra
`latitude` , `longitude`	float8	Coordenadas GPS
`ativa`	boolean	Default: true

`observacao`	text	
`deleted_at`	timestamptz	Soft delete

Rotas:

Método	Path	Query Params
GET	`/api/obras`	`clienteId`, `ativa` (true/false)
GET	`/api/obras/:id`	—
POST	`/api/obras`	Body: `CreateObraDto`
PUT	`/api/obras/:id`	Body: `UpdateObraDto`
DELETE	`/api/obras/:id`	Soft delete

5.4 Endereços de Entrega — `/api/enderecos`

Tabela: `public.enderecos_entrega` **Roles:** administrador, atendimento, gestor, fiscal

Campo	Tipo	Notas
`id`	bigserial	PK
`cliente_id`	bigint NOT NULL	FK → clientes
`obra_id`	bigint	FK → obras (opcional)
`contato`, `referencia`	text	
`telefone`, `celular`	text	
`endereco`	text NOT NULL	
`numero`, `complemento`, `cep`, `bairro`, `cidade`, `estado`	text	
`latitude`, `longitude`	float8	
`created_at`	timestamptz	

> Sem `deleted_at` — delete físico aceitável (endereços de entrega são dados operacionais reutilizáveis, mas descartáveis).

5.5 Caçambas — `/api/cacambas`

Dois níveis: **modelo** (tipo de caçamba) e **unidade** (caçamba física com patrimônio).

Tabela `cacambas` — modelo/tipo:

Campo	Tipo
`id`	bigserial PK
`nome`	text NOT NULL
`volume_m3`	numeric
`descricao`	text

Tabela `unidades_cacamba` — unidade física:

Campo	Tipo	Notas
`id`	bigserial PK	
`cacamba_id`	bigint	FK → cacambas
`patrimonio`	text	Número de patrimônio (plaqueta)
`status`	StatusCacamba	disponivel, em_uso, manutencao, descartada
`observacao`	text	

Rotas:

Método	Path
GET	`/api/cacambas`
GET	`/api/cacambas/:id`
POST	`/api/cacambas`
PUT	`/api/cacambas/:id`
GET	`/api/cacambas/unidades?cacambaId=N&status=X`
POST	`/api/cacambas/unidades`
PUT	`/api/cacambas/unidades/:id`

5.6 Veículos — `/api/veiculos`

Roles: administrador, gestor, operador

Campo	Tipo

`id`	bigserial PK
`placa`	text NOT NULL
`modelo` , `marca`	text
`ano`	integer
`capacidade_ton`	numeric
`status`	StatusVeiculo
`observacao`	text

5.7 Máquinas — `/api/maquinas`

Roles: administrador, gestor, operador

> Nota: máquinas usam `TEXT` para status (não enum no banco), mapeado para `StatusMaquina` no TypeScript.

Campo	Tipo
`id`	bigserial PK
`nome` , `tipo` , `modelo` , `marca`	text
`ano`	integer
`status`	text (StatusMaquina)
`observacao`	text

5.8 Motoristas — `/api/motoristas`

Roles: administrador, gestor, operador

Campo	Tipo	Notas
`id`	bigserial PK	
`nome`	text NOT NULL	
`cpf`	text	
`cnh`	text	Número da CNH

`validade_cnh`	date	
`categoria_a` … `categoria_e`	boolean	Uma coluna por categoria
`telefone` , `celular` , `email`	text	
`status`	StatusMotorista	
`observacao`	text	

Detalhe de implementação: O banco armazena as categorias de CNH como 5 colunas booleanas (`categoria_a` , `categoria_b` , ..., `categoria_e`). O service converte isso para um array de strings no DTO:

```
// Banco: { categoria_a: true, categoria_b: false, categoria_c: true, ... }
// DTO:   { categorias: ['A', 'C'] }
```

---

6. Mapa Completo de Rotas da API

Método	Rota	Roles	Módulo
GET	/health	público	Health check
POST	/api/auth/login	público	Auth
POST	/api/auth/refresh	público	Auth
POST	/api/auth/logout	autenticado	Auth
GET	/api/auth/me	autenticado	Auth
GET	/api/clientes	adm, atend, gestor, fiscal	Clientes
GET	/api/clientes/:id	adm, atend, gestor, fiscal	Clientes
POST	/api/clientes	adm, atend, gestor, fiscal	Clientes
PUT	/api/clientes/:id	adm, atend, gestor, fiscal	Clientes
DELETE	/api/clientes/:id	adm, atend, gestor, fiscal	Clientes
GET	/api/contatos	adm, atend, gestor, fiscal	Contatos
GET	/api/contatos/:id	adm, atend, gestor, fiscal	Contatos
POST	/api/contatos	adm, atend, gestor, fiscal	Contatos
PUT	/api/contatos/:id	adm, atend, gestor, fiscal	Contatos

DELETE	/api/contatos/:id	adm, atend, gestor, fiscal	Contatos
GET	/api/obras	adm, atend, gestor, fiscal	Obras
GET	/api/obras/:id	adm, atend, gestor, fiscal	Obras
POST	/api/obras	adm, atend, gestor, fiscal	Obras
PUT	/api/obras/:id	adm, atend, gestor, fiscal	Obras
DELETE	/api/obras/:id	adm, atend, gestor, fiscal	Obras
GET	/api/enderecos	adm, atend, gestor, fiscal	Endereços
GET	/api/enderecos/:id	adm, atend, gestor, fiscal	Endereços
POST	/api/enderecos	adm, atend, gestor, fiscal	Endereços
PUT	/api/enderecos/:id	adm, atend, gestor, fiscal	Endereços
DELETE	/api/enderecos/:id	adm, atend, gestor, fiscal	Endereços
GET	/api/cacambas	adm, atend, gestor, operador	Caçambas
GET	/api/cacambas/:id	adm, atend, gestor, operador	Caçambas
POST	/api/cacambas	adm, atend, gestor, operador	Caçambas
PUT	/api/cacambas/:id	adm, atend, gestor, operador	Caçambas
GET	/api/cacambas/unidades	adm, atend, gestor, operador	Unidades
POST	/api/cacambas/unidades	adm, atend, gestor, operador	Unidades
PUT	/api/cacambas/unidades/:id	adm, atend, gestor, operador	Unidades
GET	/api/veiculos	adm, gestor, operador	Veículos
GET	/api/veiculos/:id	adm, gestor, operador	Veículos
POST	/api/veiculos	adm, gestor, operador	Veículos
PUT	/api/veiculos/:id	adm, gestor, operador	Veículos
GET	/api/maquinas	adm, gestor, operador	Máquinas
GET	/api/maquinas/:id	adm, gestor, operador	Máquinas
POST	/api/maquinas	adm, gestor, operador	Máquinas
PUT	/api/maquinas/:id	adm, gestor, operador	Máquinas
GET	/api/motoristas	adm, gestor, operador	Motoristas
GET	/api/motoristas/:id	adm, gestor, operador	Motoristas
POST	/api/motoristas	adm, gestor, operador	Motoristas

PUT	/api/motoristas/:id	adm, gestor, operador	Motoristas
*	/api/fiscal/*	adm, fiscal, gestor, atend	Fiscal (Nathalia)
*	/api/* (financeiro)	adm, financeiro, fiscal, gestor, atend	Financeiro (Nathalia)
POST	/api/webhook/*	público	Webhook Sicoob

7. Estrutura de Arquivos Criados

```
lapaCacambas/
├── shared/
│   ├── enums.ts                        ← Fonte única de enums
│   └── contracts.md                    ← Contratos frontend ↔ backend
│
├── backend/
│   └── src/
│       ├── app.ts                      ← Registro de todas as rotas
│       ├── middlewares/
│       │   ├── auth.middleware.ts      ← requireAuth()
│       │   └── error.middleware.ts
│       └── modules/
│           ├── auth/
│           │   └── auth.controller.ts
│           ├── clientes/
│           │   ├── clientes.types.ts
│           │   ├── clientes.service.ts
│           │   └── clientes.controller.ts
│           ├── contatos/
│           │   ├── contatos.types.ts
│           │   ├── contatos.service.ts
│           │   └── contatos.controller.ts
│           ├── obras/
│           │   ├── obras.types.ts
│           │   ├── obras.service.ts
│           │   └── obras.controller.ts
│           ├── enderecos/
│           │   ├── enderecos.types.ts
│           │   ├── enderecos.service.ts
│           │   └── enderecos.controller.ts
│           ├── cacambas/
│           │   ├── cacambas.types.ts
│           │   ├── cacambas.service.ts
│           │   └── cacambas.controller.ts
│           ├── veiculos/
│           │   ├── veiculos.types.ts
│           │   ├── veiculos.service.ts
│           │   └── veiculos.controller.ts
│           ├── maquinas/
│           │   ├── maquinas.types.ts
│           │   ├── maquinas.service.ts
```

```
│           │   └── maquinas.controller.ts
│           └── motoristas/
│               ├── motoristas.types.ts
│               ├── motoristas.service.ts
│               └── motoristas.controller.ts
│
└── src/
    └── types/
        └── enums.ts                    ← Re-export de shared/enums.ts
```

---

8. Decisões de Design

Soft Delete vs Delete Físico

Tabela	Estratégia	Motivo
`clientes`	Soft delete (`deleted_at`)	Dados reais de clientes, histórico de pedidos
`obras`	Soft delete (`deleted_at`)	Pode ter pedidos associados
`contatos_cliente`	Delete físico	Dado secundário, sem referências externas
`enderecos_entrega`	Delete físico	Dado operacional sem histórico vinculado

Dois Clientes Supabase

Cliente	Chave	Uso
`supabaseAdmin`	`service_role_key`	Backend — bypassa RLS, acesso total
`supabaseAuth`	`anon_key`	Operações de auth (login, refresh)

O `service_role_key` **nunca vai para o frontend**. Fica apenas em variável de ambiente do servidor.

Auditoria

Clientes têm `created_by` e `updated_by` (UUID do usuário autenticado). O middleware injeta `req.user.id` que é passado pelos controllers para os services no momento de criar/atualizar.

---

9. Pendências e Próximos Passos

Sprint 2 — Frontend (a fazer)

- ☐ Tela de listagem de clientes
- ☐ Formulário de cadastro/edição de cliente
- ☐ Painel de disponibilidade de ativos (caçambas, veículos, máquinas)
- ☐ Tela de motoristas

Sprint 3 — Logística (a planejar com Nathalia)

- ☐ Módulo de pedidos
- ☐ Programação diária / rotas
- ☐ Módulo de execuções
- ☐ Schema definitivo da tabela `execucoes` (interface crítica com fiscal)

Itens de Infraestrutura

- ☐ CORS restrito para domínio Vercel em produção
- ☐ Trilha de auditoria completa (`logs_auditoria`)
- ☐ Vínculo motorista ↔ caminhão (tabela de associação)
- ☐ Horímetro para máquinas (campo pendente no banco)

---

10. Conformidade com os Documentos do Projeto

Documento 1 — Plano de Divisão de Trabalho

Entrega Sprint 1 (Rodrigo)	Status
Auth/RBAC	✅
Schema tabelas base	✅
CRUD Clientes	✅
CRUD Obras	✅
CRUD Endereços	✅
Shared enums	✅

Documento 2 — Documento de Nivelamento Técnico

Requisito	Status
Arquitetura types → service → controller	✅
supabaseAdmin no backend	✅

Soft delete em entidades principais	✅
RBAC via middleware	✅
snake_case → camelCase no toDto()	✅
Shared/enums como fonte única	✅
Contratos documentados (contracts.md)	✅
Contatos do cliente (seção 5.2)	✅

Documento gerado automaticamente a partir do código implementado. Para dúvidas sobre o escopo de Nathalia (fiscal/financeiro), consultar o documento de divisão de trabalho.